

PROJECT REPORT
PORT SCANNER USING NPM AND PYTHON

DURGA BASKAR
LAST UPDATED – 16/09/2024

Find my project at my GitHub repo named
[Port-Scanner](#)

Introduction

This project involves creating a port scanner using both Node.js (via npm) and Python on Parrot OS. A port scanner is a network security tool used to identify open ports and services available on a networked computer.

Objectives

- Implement a port scanner using Node.js.
 - Implement a port scanner using Python.
 - Compare the results of both implementations.
 - Document the setup, code, results, and advantages/disadvantages.
-

Setup and Environment

1. System Environment

- Operating System: Parrot OS
- Node.js Version: 18.x (or latest stable)
- Python Version: 3.x (or latest stable)

2. Software Installation

- **Node.js and npm**

```
sudo apt update  
sudo apt install nodejs npm
```

- Python 3

```
sudo apt update  
sudo apt install python3 python3-pip
```

Implementation

Node.js Implementation

1. Project Initialization

- Create Project Directory:

```
mkdir PortScanner-node  
cd PortScanner-node
```

- Initialize Node.js Project:

```
npm init -y
```

2. Install Dependencies

```
npm install net
```

3. Create Port Scanner Script

Create a file named [scanner.js](#) with the following content:

4. Run Node.js Port Scanner

```
node scanner.js
```

Python Implementation

1. Create Python Script

Create a file named [scanner.py](#) with the following content:

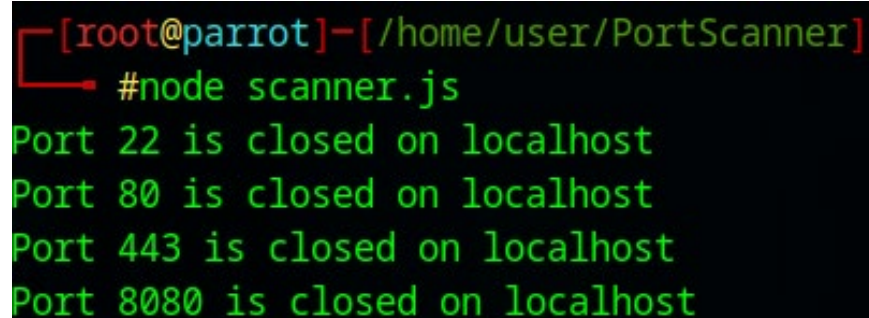
2. Run Python Port Scanner

```
python3 scanner.py
```

Results

1. Node.js Port Scanner Results

Screenshots of Results:



```
[root@parrot]-[/home/user/PortScanner]
#node scanner.js
Port 22 is closed on localhost
Port 80 is closed on localhost
Port 443 is closed on localhost
Port 8080 is closed on localhost
```

2. Python Port Scanner Results

Screenshots of Results:

```
[root@parrot]-[/home/user/PortScanner]
#python3 scanner.py
Port 22 is closed on localhost
Port 80 is closed on localhost
Port 443 is closed on localhost
Port 8080 is closed on localhost
```

Advantages and Disadvantages

1. Node.js (npm)

Advantages:

- **Asynchronous Operations:** Node.js's non-blocking I/O model allows for high performance and efficient handling of multiple simultaneous network connections.
- **Unified Language:** Using JavaScript for both server-side and client-side code can simplify development.
- **Rich Ecosystem:** The npm repository offers a wide range of libraries and tools that can be leveraged in Node.js projects.
- **Event-Driven Architecture:** Suitable for applications that require real-time data processing.

Disadvantages:

- **Callback Hell:** Asynchronous code can become complex and harder to manage, especially without proper handling of callbacks or promises.

- **Single-Threaded:** Node.js operates on a single-threaded event loop, which can be a limitation for CPU-bound tasks.
- **Limited Built-In Libraries:** Compared to Python, Node.js may have fewer built-in libraries for various specialized tasks.

2. Python

Advantages:

- **Readability and Simplicity:** Python's syntax is clear and easy to understand, which aids in development and maintenance.
- **Rich Standard Library:** Python's standard library includes many modules and functions that simplify complex tasks, including network programming.
- **Multi-Threading Support:** Python's threading model can be beneficial for CPU-bound tasks, despite the Global Interpreter Lock (GIL) limitations.
- **Versatility:** Python is widely used in various fields, including data analysis, web development, and automation.

Disadvantages:

- **Performance:** Python may be slower compared to Node.js for I/O-bound tasks due to its synchronous nature.
- **Global Interpreter Lock (GIL):** The GIL can be a bottleneck in CPU-bound multi-threaded programs.

- Less Concurrency: Python's default threading and asynchronous capabilities are less robust compared to Node.js's event-driven architecture.
-

Detailed Implementation Steps

1. Node.js Implementation

- Set up a new Node.js project by initializing it with npm.
- Install the net module to use for network operations.
- Write the port scanning logic in scanner.js to connect to each port and determine if it is open or closed.
- Execute the script using Node.js and observe the output.

2. Python Implementation

- Write the port scanning script in scanner.py using Python's socket library to attempt connections to each port.
 - Run the script using Python and check the results for open and closed ports.
-

Conclusion

Both Node.js and Python implementations successfully scanned ports and identified their status. The choice between Node.js and Python may depend on the specific use case and the environment in which the scanner will be deployed. Node.js is preferred for real-time and high-

performance scenarios due to its asynchronous nature, while Python is valued for its readability and comprehensive standard library.